

CMPS 6610 Problem Set 01 ## Answers

Name: Muhammad Suhaib Rashid

Place all written answers from `assignment-01.md` here for easier grading.

1. **Asymptotic notation**

- 1b) **Yes**

Why: Using the limit comparison test:

$$\lim_{n \rightarrow \infty} \frac{2^{n+1}}{2^n} = \lim_{n \rightarrow \infty} \frac{2 \cdot 2^n}{2^n} = \lim_{n \rightarrow \infty} 2 = 2$$

The limit is a finite constant so $2^{n+1} \in O(2^n)$

By definition: $f(n) \in O(g(n))$ if there exist constants $c > 0$ and $n_0 \geq 0$ such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.

Since $2^{n+1} = 2 \cdot 2^n$ if we choose $c = 2$ and $n_0 = 0$. Then for all $n \geq 0$:

$$2 \cdot 2^n \leq 2 \cdot 2^n$$

So the definition is satisfied. Hence $2^{n+1} \in O(2^n)$

- 1c) **No**

Why:

$$\lim_{n \rightarrow \infty} \frac{2^{2^n}}{2^n} = \lim_{n \rightarrow \infty} 2^{2^n - n}$$

As 2^n grows exponentially and n grows only linearly, $2^n - n \rightarrow \infty$ as $n \rightarrow \infty$. Therefore:

$$\lim_{n \rightarrow \infty} 2^{2^n - n} = \infty$$

so, $2^{2^n} \notin O(2^n)$.

- 1d) **No**

Why:

$$\lim_{n \rightarrow \infty} \frac{n^{1.01}}{\log^2 n}$$

Substitute $n = 2^a$ By taking log on both sides

$$\log n = a$$

$$\log^2 n = a^2$$

and

$$n^{1.01} = 2^{1.01a}$$

So limit becomes:

$$\lim_{a \rightarrow \infty} \frac{2^{1.01a}}{a^2}$$

This is an exponential over a polynomial, which is a standard divergent limit.

$$\lim_{a \rightarrow \infty} \frac{2^{1.01a}}{a^2} = \infty$$

which means $n^{1.01} \notin O(\log^2 n)$.

- 1e) **No**

Why:

$$\lim_{n \rightarrow \infty} \frac{\sqrt{n}}{\log^3 n}$$

Substitute $n = 2^a$ Square Root on both sides:

$$\sqrt{n} = 2^{a/2}$$

Taking log on both sides:

$$\log n = a$$

$$\log^3 n = a^3$$

So the limit becomes:

$$\lim_{m \rightarrow \infty} \frac{2^{m/2}}{m^3}$$

This is an exponential over a polynomial so:

$$\lim_{m \rightarrow \infty} \frac{2^{m/2}}{m^3} = \infty$$

i.e. $\sqrt{n} \notin O(\log^3 n)$.

- 1f) **Yes**

Why:

$$\lim_{n \rightarrow \infty} \frac{\sqrt{n}}{\log^3 n} = \infty$$

As shown in question (1e) the limit is ∞ , so

$$\sqrt{n} \in \Omega(\log^3 n)$$

- 1g) The definitions for small o and small omega are:

$$f(n) \leq c \cdot g(n), \text{ for all } c$$

$$f(n) \geq c \cdot g(n), \text{ for all } c$$

Let's assume there is an $f(n)$ that satisfies both conditions. So:

$$c \cdot g(n) \leq f(n) \leq c \cdot g(n), \text{ for all } c$$

This basically means:

$$f(n) = c \cdot g(n), \text{ for all } c$$

But this must hold for every positive constant c since $f(n)$ is one fixed function. So it must be true for example for $c = 1$ and for $c = 2$ simultaneously:

$$f(n) = 1 \cdot g(n)$$

$$f(n) = 2 \cdot g(n)$$

Combining these:

$$g(n) = 2 \cdot g(n)$$

dividing both sides by $g(n)$ gives:

$$1 = 2$$

This is a contradiction.

Therefore, no such $f(n)$ exists and $o(g(n)) \cap \omega(g(n))$ is an empty set.

2. SPARC to Python

- 2b) `foo(a, b)` computes the GCD of `a` and `b`. It repeatedly replaces the larger number with the remainder of dividing it by the smaller number, until one of them hits 0, then the other number is the GCD.
- 2c) Each call to `foo` does constant work and makes exactly one recursive call, so the whole computation is sequential which means $Work = Span$.

$$Work = Span = O(\log n)$$

3. Parallelism and recursion

- 3b) A single loop scans all n elements once doing constant work per iteration. So:

$$Work = O(n), Span = O(n)$$

- 3d) Each call splits the array in half, recurses on both halves, then does constant work combining the two results. Since no recursive call runs in parallel, work and span are the same:

$$Work = 2W(n/2) + O(1) = O(n)$$

$$Span = 2S(n/2) + O(1) = O(n)$$

- 3e) Total work doesn't change as the same number of operations happen:

$$Work = 2W(n/2) + O(1) = O(n)$$

But now the two recursive calls run concurrently, so the span only accounts for the slower of the two branches not both added together:

$$Span = \max(S(n/2), S(n/2)) + O(1) = S(n/2) + O(1) = O(\log n)$$

4. GCD